Software: Practice and Experience

WILEY

RESEARCH ARTICLE

K8sidecar: A Modular Kubernetes Chain of Sidecar Proxies for Microservices and Serverless Architectures

Andoni Salcedo-Navarro | Miguel Garcia-Pineda | Juan Gutiérrez-Aguado

Departamento de Informática, Universitat de València, Burjassot (Valencia), Spain

Correspondence: Juan Gutiérrez-Aguado (juan.gutierrez@uv.es)

Received: 12 November 2024 | **Revised:** 12 February 2025 | **Accepted:** 19 March 2025

Funding: This study was supported by grant PID2021-126209OB-I00 funded by MCIN/AEI/10.13039/501100011033 and by "ERDF A Way of Making Europe".

Keywords: kubernetes | microservices | operator pattern | proxy sidecar | serverless computing

ABSTRACT

Background: Modern microservice architectures demand not only modularity, scalability, and maintainability but also adaptation to dynamic requirements. For applications deployed on Kubernetes, sidecar proxies have been used to separate the operational features (such as security, networking, or monitoring) from the application business logic by intercepting traffic to and from microservices or functions in serverless platforms. Existing proxy sidecar implementations such as Envoy offer these operational features as chains of logic that cannot be composed at deployment time.

Objective: This work introduces K8Sidecar, which leverages the operator pattern to dynamically integrate a chain of proxy sidecar containers into microservices or serverless architectures deployed on top of Kubernetes.

Methods: The study presents the architecture of the software and describes the Java and Go libraries provided to develop new proxy sidecars.

Results: We provide illustrative examples and evaluate the impact of the number of injected sidecars on cold-start and latency and compare them with Envoy.

Conclusion: Results show that for up to 5 sidecars, the chain-of-sidecars approach (especially using the Go library) remains reasonably close to Envoy in terms of both cold start and latency.

1 | Introduction

The last decade has been marked by a rapid transition from monolithic architectures to microservices and ephemeral replicas of functions. While this shift provides modular, scalable, and fault-tolerant systems, it also raises a new set of challenges. There are two opposing requirements, on the one hand, individual services must be lightweight and single-purposed. On the other hand, they must integrate functionalities like logging, monitoring, or security, which are an integral part of achieving maintainable and observable systems [1].

Among the various patterns utilized in cloud-native architectures, the proxy sidecar pattern has been identified as a potential solution to this problem [2]. The proxy sidecar is a design pattern intended for containers that cooperate closely and that are deployed on the same pod [3]. The sidecar proxy pattern has been widely adopted to implement a service mesh that provides unified control and observability with fine granularity. Acting as complementary auxiliary containers in the data plane, proxy sidecars allow functionalities to be abstracted away from the application, ensuring that microservices adhere to the single responsibility principle. However, a lack of a dynamic and

intuitive method for the chaining of sidecars at deployment time has posed a challenge for their use in Kubernetes or Serverless architectures [4].

A recent study on monitoring tools for DevOps [5] identified key research challenges. These include assessing the runtime overhead of monitoring-based systems and assessing the risk of introducing bugs through instrumentation code in microservices. Therefore, a good practice could be to externalize the monitoring code to a proxy sidecar that can be tested in isolation and its runtime impact can be clearly measured.

Envoy [6] is one of the most known and widely used proxy sidecar implementations. Envoy proxy intercepts traffic enabling security policies, rate limiting, and other transformations. Besides, it offers dynamic routing, load balancing, service discovery, or protocol support. Envoy can be extended by adding custom filters for specific tasks. However, these filters must be developed and integrated into a filter chain during the development phase; therefore, it is not possible to combine filters or add more filters dynamically in a chain when deploying an application.

The proposed solution addresses the challenge of seamlessly integrating and chaining proxy sidecars into Kubernetes environments [7] by dynamically injecting them by using the operator pattern. This approach bridges the gap between the necessity of adding extra common functionalities and the goal of maintaining the simplicity of microservices. Manual integration of sidecars can lead to inconsistencies and substantial maintenance overheads. By automating this process, our solution offers a consistent, scalable, and modular approach to sidecar chaining.

From the user's perspective, the proposed solution provides a smooth experience. Users can implement custom sidecars such as logging, monitoring, authentication, data management, adapters, etc. by using our libraries. These sidecars can be tested and optimized in isolation. The chain of sidecars, or filter, is deployed as a custom resource, and when a deployment or a serverless function is labeled to use the filter, the system automatically injects the required sidecars into the respective microservices or functions. This abstraction not only reduces the need for manual intervention, but also ensures that the primary focus remains on developing the core service logic.

The main contributions of this proposal are:

- *On-the-fly configuration*: Enables the addition, removal, or update of a chain of proxy sidecars without the need to rebuild or redeploy the core microservice or function. This functionality supports rapid iteration and deployment cycles, significantly reducing downtime and increasing development agility.
- *Custom Resource Definition (CRD) for Sidecars*: Leverages Kubernetes CRDs for dynamic chain configurations, enabling developers to define individual sidecar properties such as image, name, environment variables, or volumes. This allows for customized functionality and behavior within the microservices architecture.
- *Label-Based Injection Control*: Utilizes Kubernetes labels to control the sidecar injection into specific deployments, providing granular control over which microservices are paired with certain sidecars, thus enhancing the architecture's modularity and flexibility.
- *Priority-based injection*: Allows developers to assign priorities to sidecars at deployment time, dictating the order in which they are injected and executed. This ensures that functionalities, such as security or logging, are processed in the desired order, optimizing the flow of requests or dealing with dependencies through the chain.
- *Environment variables for configuration*: Allows the injection of Kubernetes environment variables into sidecars, enabling them to adjust seamlessly to different deployment environments. This feature is useful for configuring sidecars to operate under various conditions without modifying their core logic.
- *Golang and Java Libraries*: Offers dedicated libraries for Golang and Java, simplifying the development of new custom sidecar functionalities by abstracting sidecar integration complexities. This allows developers to focus on implementing the specific functionality of the sidecar.
- *Simplified Middleware Function Definition*: The libraries provide interfaces for defining middleware functions that manage HTTP requests and CloudEvents, accommodating both traditional web applications and event-driven architectures. This versatility supports a wide range of use cases and application scenarios.
- *Knative Support*: Enhances serverless computing models managed by Knative, facilitating sidecar injection in serverless frameworks. This feature enables developers to utilize dynamic sidecar management benefits in cloud-native applications that scale automatically based on demand.
- *Event-Driven Architecture Compatibility*: With CloudEvents support, the software facilitates the development of event-driven microservices, ensuring the integration among decoupled services.

The remainder of the paper is organized as follows: Section 2 provides an overview of the technical stack used in this study, including Kubernetes, the Function-as-a-Service model, and the Operator Pattern. Section 3 presents a high-level overview of the proposed solution. Section 4 offers a detailed description of the various components and libraries available for filter development. Validation using Kubernetes resources of type `Deployment` and Knative resources of type `Service` (for serverless functions), performance experiments, and a comparison with related work are discussed in Section 5. Finally, the conclusions and future work are presented in Section 6.

2 | Background

This section briefly presents the technological stack upon which the proposal has been developed.

2.1 | Kubernetes

Kubernetes [7] is an open-source container orchestration platform that has emerged as the *de facto* standard for managing and orchestrating containerized workloads. Initially designed by Google and now maintained by the Cloud Native Computing Foundation (CNCF), Kubernetes provides a powerful and flexible way to automate deployments, scale applications, and ensure high availability through self-healing mechanisms. At a high level, Kubernetes offers a REST API to manage resources such as nodes, pods, services, volumes, deployments, replicas, etc., that can be used to run containerized applications. Nodes represent the physical or virtual machines where containers can be allocated. Pods are units that encapsulate one or more containers, which are deployed together and share the same network and storage. Services provide stable IP addresses and DNS names for pods. Deployments define the desired state of a group of replicas, with each replica running a pod, ensuring redundancy and availability. Kubernetes also provides advanced features such as rolling updates, blue-green deployments, and horizontal scaling.

2.2 | Function-As-A-Service

Function as a Service (FaaS), a key component of serverless computing, represents a new approach to building and running applications in the cloud [8]. In traditional architectures, developers are responsible for managing the infrastructure where the service will be deployed. This includes the management of elasticity to adapt the underlying infrastructure to the demand. With FaaS, instead of managing servers or containers, developers focus solely on writing and deploying individual functions that respond to specific events or requests, transferring the responsibility of the allocation and scaling of these functions to cloud providers.

Cloud providers have widely adopted this technology: AWS Lambda, Google Cloud Functions, Azure Functions, and others manage the underlying infrastructure, automatically provisioning resources and executing functions in response to events from other services or user requests. The FaaS platform will handle the scaling required to run that function to deal with the demand. For on-premises, different serverless platforms can be deployed on top of Kubernetes such as Knative or OpenWisk.

Overall, FaaS offers a granular pay-per-use model and simplifies application design, development, and deployment by abstracting away the underlying infrastructure, allowing developers to focus on writing efficient, event-driven code.

2.3 | Operator Pattern

In Kubernetes, the Operator pattern is a powerful approach to manage and deploy complex applications [9]. An Operator allows the definition of new custom resources and controllers for managing specific applications or workloads, extending the set of resources provided by Kubernetes. The Operator pattern provides a standard way to create, configure, and manage instances of these applications. This can include starting and stopping containers, scaling resources, handling backups and restores, and even automatically managing rolling updates and rollbacks.

By leveraging the Operator pattern, the need for manual intervention in complex environments is reduced. Additionally, Operators can be shared within organizations or the community to enable consistent deployment and management of applications across multiple clusters.

3 | High-Level Overview of the Chain of Sidecars Proposal

The architecture rests upon its modular and dynamic design. It is built around the primary principle of *on-the-fly* sidecar injection. Instead of incorporating all non-core logic within the main service, sidecars are treated as peripherical components that can be attached at deployment time to other resources.

The proposed solution integrates within the Kubernetes ecosystem through the development of an operator that consists of a CRD and a controller. A new `Filter` CRD has been defined, which allows the specification of an array of proxy sidecars (each filter can be tailored to a specific use case with the sidecars needed). Each sidecar within this array is distinctly identified by a unique name and a specific container image. Additionally, it is possible to define optional properties for each sidecar, such as environment variables, volumes, or priority settings. Conversely, the controller is responsible for creating a Mutating Admission Controller and deploying its associated webhook, which is tailored to a particular `Filter`. This process, illustrated in Figure 1, allows the specification of sidecar configurations by merely creating a Custom Resource of the type `Filter`. The dynamic injection of the sidecars specified in a `Filter` can be achieved by referencing the filter name using Kubernetes labels.

When a Kubernetes resource that makes use of a `Filter` is deployed, the associated webhook modifies the request to inject and configure the array of containers into the resource. The mutated request is finally processed by the kubelet on a node, which creates the pod containing the sidecars along with the application, as illustrated in Figure 2.

4 | Implementation Details and Usage

In this section we provide a detailed description of the architecture and functionalities of our software, emphasizing its design and capabilities of sidecar integration.

4.1 | Filter Operator: Custom Resource Definition and Controller Components

The CRD `Filter` allows the specification of an array of sidecars, where each sidecar has a name and uses a container image.

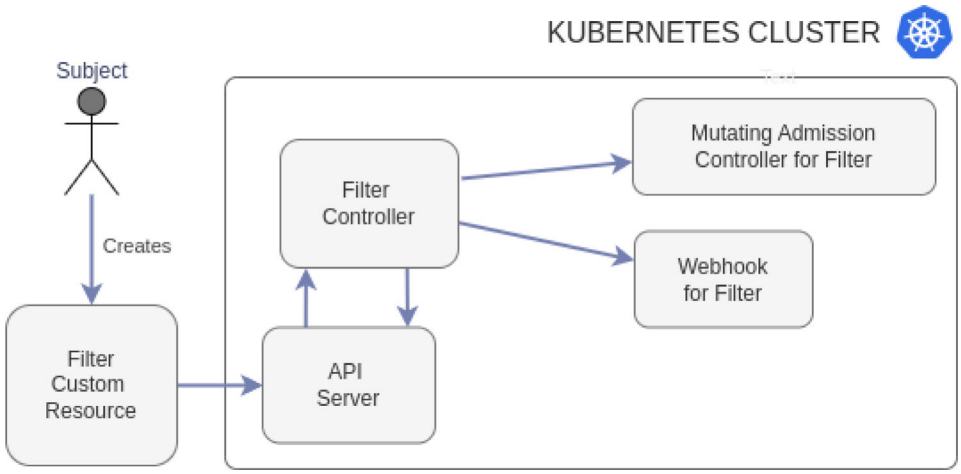


FIGURE 1 | The operator controller is notified when a new Filter is created. A new Mutating Admission Controller and its webhook are automatically created for that Filter.

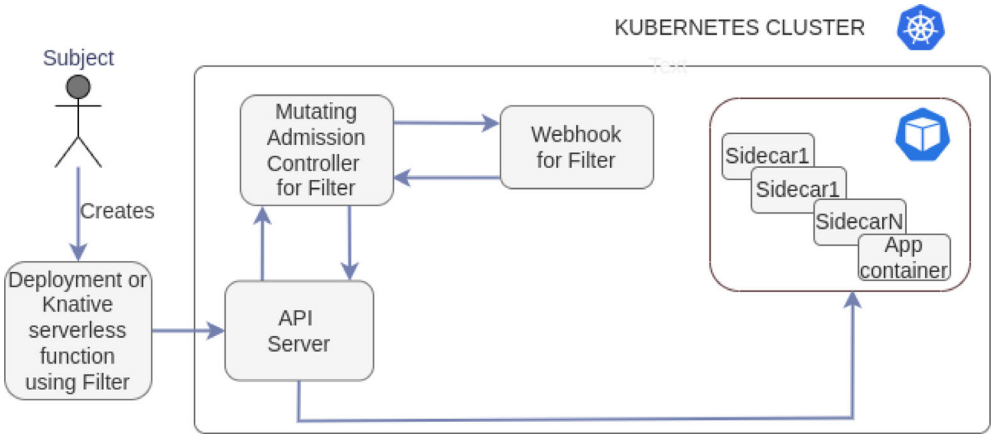


FIGURE 2 | When a resource that makes use of the Filter is deployed the webhook injects the sidecars specified in the Filter.

Additionally, each sidecar can have a priority and environment variables.

Listing 1 shows the allowed fields of each sidecar in a `Filter`, where:

- `name`: serves as the unique identifier of the sidecar within the filter. It is crucial for managing and referencing individual sidecars.
- `image`: represents the container image with the implementation of the sidecar, defining its operational environment and capabilities.
- `priority`: an optional attribute that facilitates the sorting of sidecars by assigning a priority level ranging from 0 to 255. This feature allows users to determine the order in which requests are processed by different sidecars. By default, this priority is set to 0, indicating the highest priority level.
- `Env`: this option enables the injection of environment variables into the sidecar container, providing a means to adapt the behavior of the sidecars to each deployment.
- `Vol`: volume specification for a sidecar. Each sidecar can mount its own volume.

LISTING 1: Allowed fields in the declaration of each sidecar in a Filter.

```
properties:
    name:
        type: string
    image:
        type: string
    priority:
        type: integer
        format: int8
        nullable: true
    env:
        type: Env
    vol:
        type: Volume
```

The other key component of the operator is a custom *controller* that receives the request when a new `Filter` is created or deleted. The controller, implemented in Golang[1], executes a control loop to synchronize the desired and actual states of the cluster. If a `Filter` is created, the controller creates and registers a *Mutating Admission Controller*, whose webhook will perform the injection of the sidecars if the deployed resource has a label with the name of the filter.

The steps performed by the webhook when a resource using the `Filter` is deployed are shown in Algorithm 2. The webhook receives the request and returns a `JSONPatch` with all the changes made to the original resource in the request. The priority in the sidecars determines the order of injection, ensuring that sidecars with higher priority are closer to the application container. The container with the application gets the maximum priority so it will be the last one in the generated chain. Besides, an `emptyDir` volume for temporary data sharing is created and mounted in all the containers as illustrated in Figure 3. Sharing a volume across all the containers in the pod can be useful in some scenarios. For instance, downloading data required by the application or uploading data produced can be managed by a sidecar.

ALGORITHM 2 | Algorithm used to inject sidecars specified in a Filter.

Input: resources;
Input: filter_name from environment
Input: filter_data from environment
  if resource has label matching the filter_name **then**
    port = user container port from annotation;
    mount_dir = mount point from annotation
    list = containers in resource
    sidecars = sidecars in filter
    set_pritority(user_container,MAX_PRIORITY)
    sorted_list = sort_by_priority(sidecars.append(list))
    patch = [ ]
    patch.append_operation(Delete list of containers);
    patch.append_operation(Add emptydir volume);
    while *C* in *sorted_list* **do**
      patch.append_operation(Add *C* to containers array);
      patch.append_operation(Assign *port* to C);
      patch.append_operation(Mount volume in *mount_dir*)
      port ← *port* + 1;
    end while
  end if
Output: response with *patch*;

4.2 | Libraries to Develop New Proxy Sidecars

The intended behavior for each proxy sidecar in a chain is as follows: when the sidecar receives an HTTP request it performs some action, then sends the HTTP request to the next sidecar (or the application container if the sidecar is the last one in the chain), and finally can perform some action when the callee returns. This can be implemented in any language capable of dealing with HTTP requests.

By providing libraries that simplify the process of defining the middleware function that handles requests, the development of new proxy sidecars is facilitated. We have implemented libraries in Golang and Java, following a common specification that leverages the specific features and mechanisms of each language. The implementation of the libraries follows the idea of filter chains in `Servlets`.

The libraries handle standard HTTP requests and `CloudEvents`, offering two types of functions `TriFunction` and `QuaFunction` respectively. The `TriFunction` is designed to handle standard HTTP requests, whereas the `QuaFunction` is designed to process CloudEvents, extending the applicability to event-driven architectures.

Although we provide ready-to-use libraries in both Java and Go, developers are not constrained to these languages. Indeed, creating a functional sidecar in any language compatible with the system is straightforward: the sidecar only needs to listen for HTTP requests on the port defined by the `PPORT` environment variable and then, once any intended processing is completed, forward the request to `PPORT + 1` to pass it to the next filter in the chain. This design ensures that users can adapt the pattern to their existing technology stacks and leverage the wide range of libraries or dependencies available in different ecosystems.

4.2.1 | Golang Library

The Golang library developed[2] uses the standard HTTP library due to its performance, wide adoption, and extensive support within the Golang ecosystem. Listing 3 provides the definitions of `TriFunction` and `QuaFunction` for a Golang implementation.

FIGURE 3 | The webhook creates and mounts an `emptyDir` volume shared by all the sidecars and the user container. This allows the sharing of data among all the containers in the pod.

LISTING 3: Interfaces defined in the Golang library.

```
// TriFunction is a type representing a function that takes in
// an HTTP request, an HTTP response writer, and a FilterChain.
type TriFunction func(req *http.Request,
    res http.ResponseWriter, chain *FilterChain)

// QuaFunction is a type representing a function that takes in an HTTP
// request, an HTTP response writer, a cloud event, and a FilterChain.
type QuaFunction func(*http.Request,
    http.ResponseWriter, cloudevents.Event, *FilterChain)
```

4.2.2 | Java Library

The provided Java library[3] conforms to the Servlet specification, a widely accepted standard for web application development in the Java ecosystem. A class `SidecarFilter` with the method `accept` is implemented in the library (this class uses Jetty to start an HTTP server). The `accept` method expects an object that can be referenced by one of the functional interfaces (so lambda functions can be used) shown in Listing 4.

LISTING 4: Functional interfaces defined in the Java library. The actions to be performed in the sidecar are implemented in the accept method.

```
/**
 * A functional interface representing a function that accepts three arguments.
 * For a sidecar the first argument is a HttpServletRequest,
 * the second argument is a HttpServletResponse, and the third one
 * is of type FilterChain to pass the request to the next container in the chain
 */
@FunctionalInterface
public interface TriFunction<T, U, V> {
  void accept(T t, U u, V v);
}
/**
 * A functional interface representing a function that accepts four arguments.
 * For a sidecar the first argument is a HttpServletRequest,
 * the second argument is a HttpServletResponse, the third one
 * is of type CloudEvent, and the last one is of type FilterChain
 * to pass the request to the next container in the chain
 */
@FunctionalInterface
public interface QuaFunction<T, U, V, R> {
  void accept(T t, U u, V v, R r);
}
```

As an illustrative example of usage, Listing 5 shows the use of the `QuaFunction` to develop a sidecar using the provided `SidecarFilter` class.

LISTING 5: Sample usage of QuaFunction to develop a sidecar.

```
// SidecarFilter is provided by the Java library
SidecarFilter server = new SidecarFilter((req, res, event, chain) -> {
      try {
        // Use req and event to obtain information
        // Do some action before passing the request to the next element in the chain
```

```
        chain.next();
        // Do some action before returning to the previous element in the chain
      } catch (Exception e) {
      }
    });
```

4.3 | Custom Resource to Define a Filter With a Chain of Proxy Sidecars

Let's assume that the user has developed a set of sidecars $\{S_1, S_2, \ldots, S_n\}$ using the provided libraries and wants to apply some of them to a Kubernetes deployment. A *Custom Resource* of type `Filter`, that conforms to the Custom Resource Definition explained in Subsection 4.1, must be specified and deployed into Kubernetes. An example of deployment is shown in Listing 6.

LISTING 6: YAML specification of a new Filter. Each Filter defines an array of proxy sidecars and the priority defines the order.

```yaml
kind: Filter
metadata:
  name: <filtername>
spec:
  sidecars:
    - image: <sidecar-image1:tag>
      name: sidecar_name1
      priority: 1
    - image: <sidecar-image2:tag>
      name: sidecar_name2
      priority: 2
```

As mentioned previously, when this custom resource is created, the controller dynamically creates and configures a Mutating Admission Controller with its webhook for this filter. This is illustrated in Figure 4.

4.4 | Using the Filter in a Deployment or a Serverless Function

The association of a filter with a deployment or a serverless function is achieved by using labels. In our system, the name of the filter serves as the key of the label, and the value is set to the string `sidecar`. This approach permits the injection of one or more filters into the deployment, based on the specified criteria. The webhook must know the port of the user container, for this purpose, a new annotation is used. This annotation holds the name of the environment variable that stores the port. This design allows accommodating the sidecar to existing deployments that use an arbitrary name to pass the port to the application.

Listing 7 illustrates de use of a filter in a Kubernetes deployment.

LISTING 7: Sample usage of a filter in a Kubernetes deployment.

```yaml
apiVersion: apps
kind: Deployment
metadata:
  name: app-deployment
  annotations:
    k8sidecar.port.env-name: "APP_PORT"    # default: PORT
    k8sidecar.volume.mount-dir: "/shared-data"    # default: /shared
  labels:
    app: app-label
    <filtername>: "sidecar" # Enable injection of sidecars defined in the filter
spec:
  selector:
    matchLabels:
      app: app-label
  template:
```

```
spec:
  containers:
  - name: <name>
    image: <user-container-image:tag>
    env:
      - name: APP_PORT
        value: <port>
    ports:
    - containerPort: <port>
```

When this resource is deployed in Kubernetes, the webhook of the mutating admission controller is activated to inject the sidecars. This process is shown in Figure 5.

Listing 8 illustrates de use of a filter in a Knative serverless function. As can be seen, the syntax to use the filter is identical to that used for Kubernetes deployments.

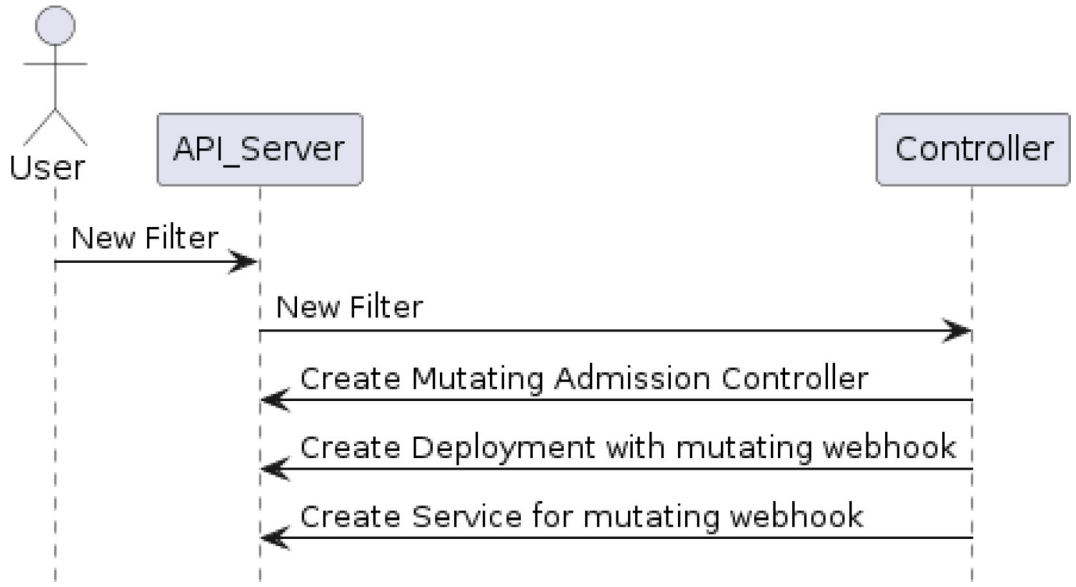


FIGURE 4 | When the user creates a new Filter, the controller automatically creates a new Mutating Admission Controller with its webhook.

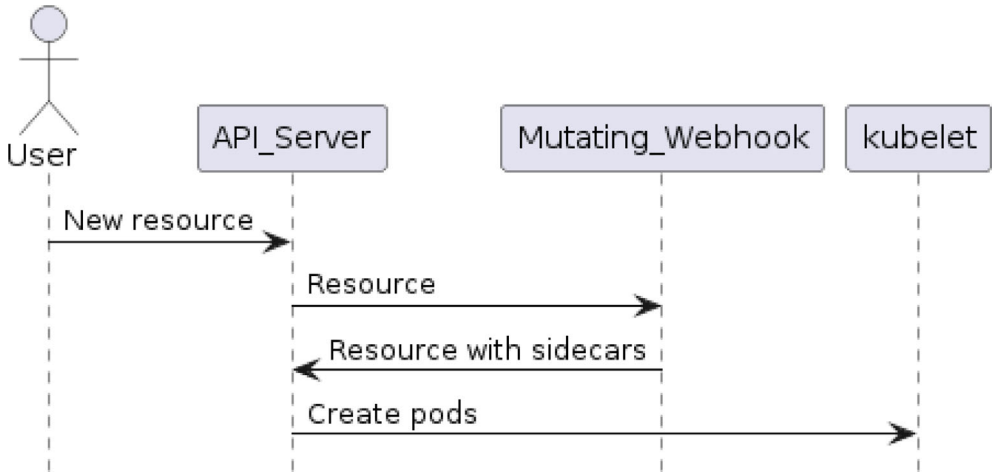


FIGURE 5 | When the user creates a new Filter, the controller automatically creates a new Mutating Admission Controller with its webhook.

LISTING 8: Sample usage of a filter in a Knative function.

```
kind: Service
apiVersion: serving.knative.dev/v1
metadata:
  name: serverless-fn
  annotations:
    k8sidecar.port.env-name: "APP_PORT"    # default: PORT
    k8sidecar.volume.mount-dir: "/shared-data"    # default: /shared
  labels:
    <filtername>: "sidecar"
spec:
  template:
    metadata:
      name: serverless-fn-v2
    spec:
      containers:
        - name: <name>
          image: <user-container-image:tag>
          env:
            - name: APP_PORT
              value: <port>
          ports:
          - containerPort: <port>
```

5 | Validation and Performance Evaluation

5.1 | Validation

This section provides a comprehensive overview of the diverse range of sidecars that have been implemented, deployed, and subjected to rigorous testing on our platform. It encompasses a multitude of essential functionalities, including security, data management, fault tolerance, and monitoring. Each sidecar is designed to address a specific operational need without modifying the core application code, thereby enhancing the system's modular architecture.

- *Data Management*: Facilitates the efficient handling of data, enabling both the downloading and uploading of data consumed or produced by the application. This sidecar ensures data synchronization across distributed components.
- *CloudEvent Generation*: Converts internal system events into standardized CloudEvents, thereby promoting interoperability across different cloud-based services and platforms.
- *Rate limiting*: Limits the number of requests to a server[4].
- *Logging*: Captures and stores logs systematically, aiding in troubleshooting and maintaining the system's operational integrity over time[5].
- *Token Authorization*: Ensures secure service-to-service communication through token checks, enforcing authentication and authorization[6].
- *Event Adapter for RabbitMQ*: This sidecar acts as a middleware that subscribes to RabbitMQ queues to retrieve messages, which are then transformed into CloudEvents and forwarded as HTTP requests to subsequent services in the chain. This decouples the application from specific message queue technologies.
- *GPU Usage Monitoring*: Monitors and logs GPU usage statistics, providing insights for resource optimization and system performance tuning.

To illustrate the practical application of these sidecars, consider a scenario where an application is developed to consume CloudEvents wrapped in HTTP requests. Traditionally, if the event data is stored in RabbitMQ, adapting the application to read directly from RabbitMQ would necessitate substantial code modifications. However, with the implementation of *Event Adapter for RabbitMQ* sidecar, there is no need to alter the application's code. The sidecar integrates with RabbitMQ, fetching messages, converting them into the appropriate format, and ensuring they are correctly injected into the application flow. This not only simplifies the architecture but also enhances its adaptability and scalability, demonstrating the sidecar's utility in practical cloud-native environments. This integration process is illustrated in the diagram shown in Figure 6.

5.2 | Performance Evaluation

This section describes the testbed and the experiments performed. The first experiment evaluates the cold start time in serverless platforms. Cold start refers to the time taken for a service to become available to handle requests when the number of replicas is zero. We measured this time as the elapsed time from the moment a request was sent to the instant when the user application received the request. This experiment has

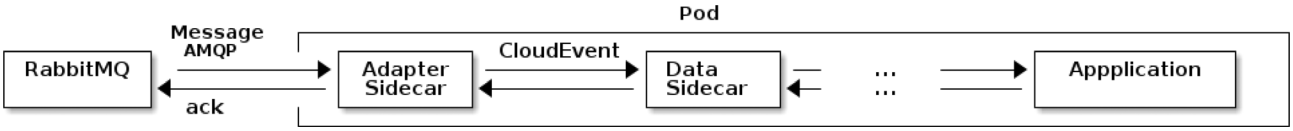
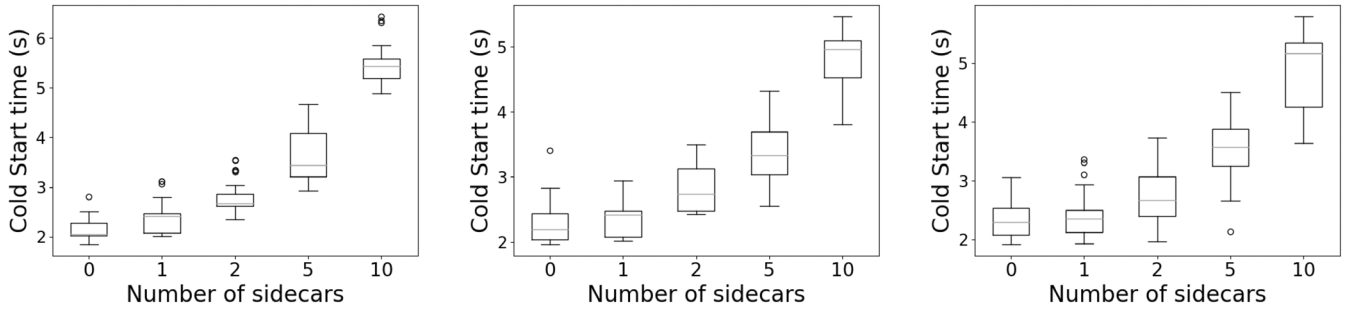


FIGURE 6 | Workflow diagram of the RabbitMQ adapter sidecar integration.



a) 8 requests b) 24 requests c) 72 requests

FIGURE 7 | Cold-start comparison depending on the number of proxy sidecars, implemented using Go, per pod. In the more demanding scenario (72 requests and 10 sidecars) each worker node has to start 108 containers (each pod has the queue-proxy, 10 sidecars, and the user container).

been performed with small and big sidecars implemented in Go and Java, respectively. We also want to measure the impact of the number of proxy sidecars on the latency. We measured the latency as the duration of time that has elapsed since the initial receipt of the request in the first proxy, the requests pass through all the subsequent proxy sidecars, ultimately arriving at the user container, and returning to the original proxy that initially received the request. This experiment was also performed with two sidecars implemented in Go and Java using the provided libraries.

5.2.1 | Test Bed

In the experimental setup we used three servers interconnected via a 10 Gb/s network. Each server hosts virtual machines (VMs) configured with SR-IOV enabled NICs, assigning a Virtual Function (VF) to each VM; the Kubernetes cluster consists of a master VM with 12 vCPUs and 32 GB RAM, and eight worker VMs each equipped with 10 vCPUs and 24 GB RAM, running Kubernetes version 1.29 and Knative version 1.14.

5.2.2 | Impact of the Number of Small Sidecars in the Cold Start Time

In this experiment, we will evaluate the impact of the number of sidecars in the cold start time, measured as the time that elapses from when the request is sent until it reaches the application container and can be attended. As we have 8 worker nodes, and a constrained platform in terms of resources, we performed three tests:

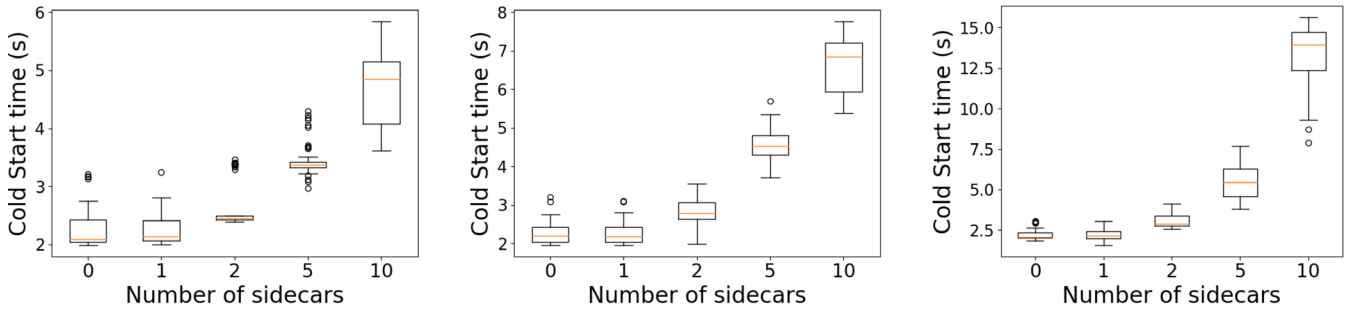
- 8 simultaneous calls ensuring that each replica lands in a different worker node (low demand)
- 24 simultaneous calls so that each worker node allocates three replicas (medium demand)
- 72 simultaneous calls with 9 replicas per worker node (high demand)

Besides, this experiment was repeated across different scenarios, specifically with deployments containing 0, 1, 2, 5, and 10 sidecar containers.

In the most demanding scenario, each worker node will be requested to start 108 containers (9 pods with 12 containers: the queue-proxy injected by Knative, the 10 proxy sidecars, and the user container). In particular, we used a sidecar implemented in Go that simply forwards the request to the next proxy sidecar. The container image of this sidecar is 17.2 MB. The queue-proxy image has 164 MB, and the user image (in this experiment an application with an echo) has 125 MB.

The experiment has been repeated 100 times to account for variability and transient environmental conditions that could affect the results. During this experiment no other workload was executed in the platform. Figure 7 shows the boxplot of the distributions of response times across different numbers of sidecar containers. From the data we can conclude, as expected, that as more sidecars are included, the cold start time increases. Therefore, for deployments where cold start performance is critical the number of sidecars should be carefully considered. While sidecars add valuable capabilities, their impact on startup time can be detrimental to the user experience in latency-sensitive applications. Results show that the cold start mean overhead increases a 66% with 5 proxy sidecars compared to the base case when each worker node has to start one replica of the pod and rises to 103% when each worker node has to start 9 replicas of the pod. We consider that use cases with more than five proxy sidecars are unlikely.

Another consequence of this data is that the number of simultaneous requests performed has not a big impact. This can be due to the fact that the start time is dominated



a) 8 requests b) 24 requests c) 72 requests

FIGURE 8 | Cold-start comparison depending on the number of proxy sidecars, implemented using Java, per pod. In the more demanding scenario (72 requests) each worker node has to start 108 containers.

by the two bigger containers (the queue-proxy and the user container).

5.2.3 | Impact of the Number of Big Sidecars on the Cold Start Time

In this case, we have used the Java library to implement a sidecar that, as in the previous section, simply forwards the request to the next element in the chain. The image of this proxy sidecar is 62.4 MB. Using this proxy sidecar, we repeated the same tests described in the previous section.

Figure 8 shows the boxplot of the distributions of response times across different numbers of sidecar containers.

As in the results with the Go implementation, we conclude that as more sidecars are included, the cold start time increases. This overhead is more evident when the demand to start more pods simultaneously increases. With 24 simultaneous requests, the mean overhead to start 10 sidecars is 4.3282 s (from 2.2558 s with no proxy sidecars to 6.5959 s with ten proxy sidecars). With 72 simultaneous requests, the mean overhead of using 10 sidecars increases the time by 10.9131 s (from 2.2027 to 13.1158 s).

5.2.4 | Impact of the Number of Sidecars and Implementation Language on Latency

The objective of this experiment was to quantify the impact of sidecar containers on the latency of requests processed by microservices. Specifically, the objective was to quantify the time taken from when a request is sent to a server until it is received by the core application, across varying numbers of sidecars when the sidecars merely pass the request to the subsequent element in the chain. In this case, the focus was on the impact of the network stack on latency, given that the packets must traverse this network stack at each sidecar proxy.

The latency experiment involved sending 600 requests to a server configured with different numbers of sidecars (0, 1, 2, 5, and 10).

Latency was measured in milliseconds, capturing the duration from the transmission of the request to its receipt by the core application. This approach allowed us to assess the additional latency attributable to the presence of sidecar containers. Figure 9 illustrates the distribution of request latency for each configuration across different numbers of sidecar containers and implementations.

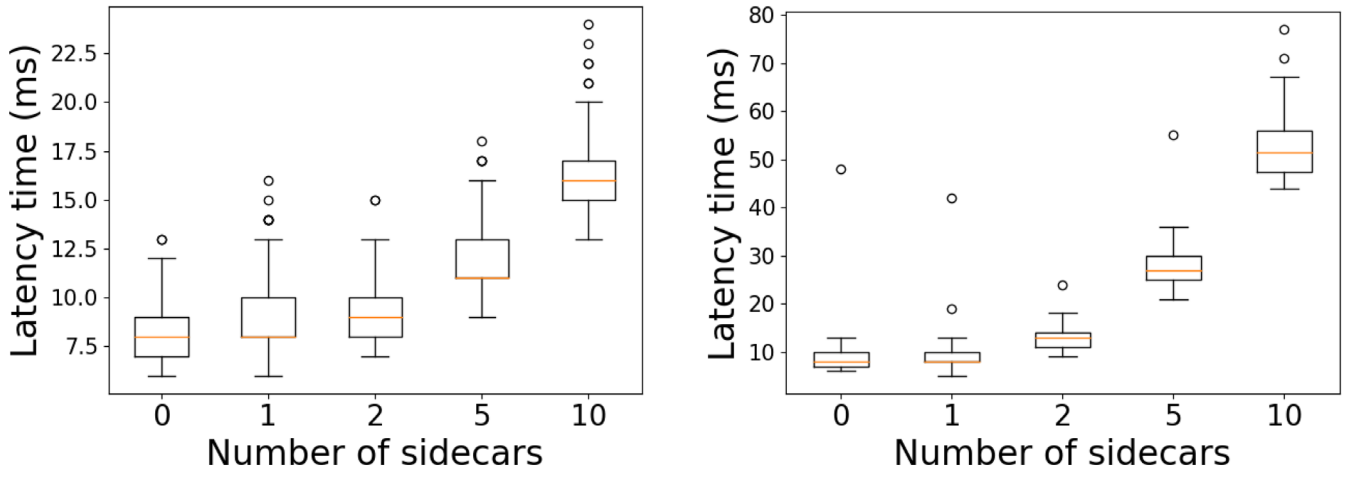
The impact of sidecars on latency within microservices architectures demonstrates that deployments with a single sidecar result in minimal latency.

As the number of sidecars increased, a gradual rise in latency was observed. As illustrated in the optimal scenario depicted in Figure 9a, the mean latency increased from 7.80 ms with one sidecar to 9.30 ms with two sidecars and to 12.40 ms with five sidecars. The deployment with ten sidecars exhibited a notable increase in latency, with a mean latency of 16.11 ms. This indicates a total latency addition of 7.51 ms from the baseline to deployment with ten sidecars. Despite the progressive increase in latency with each additional sidecar, even with ten sidecars, the resulting increase remains modest. This increase is negligible when considered in the context of typical usage patterns in most applications, which would not typically involve the deployment of ten sidecars.

5.2.5 | Performance Comparison With Envoy

Envoy is a well-established single-proxy solution widely used in production due to its comprehensive set of built-in filters and relatively low overhead when managing standard functionalities (e.g., load balancing, telemetry, security). However, our chain-of-sidecars approach targets a different set of objectives. We allow developers to implement sidecars in any programming language, integrate any required libraries, and enable specialized use cases (e.g., data meshes that download data to a shared filesystem before handing it off to the main application). These customization needs are harder to achieve with Envoy filters, which require C++ or WebAssembly development and recompilation or complex plugin integration.

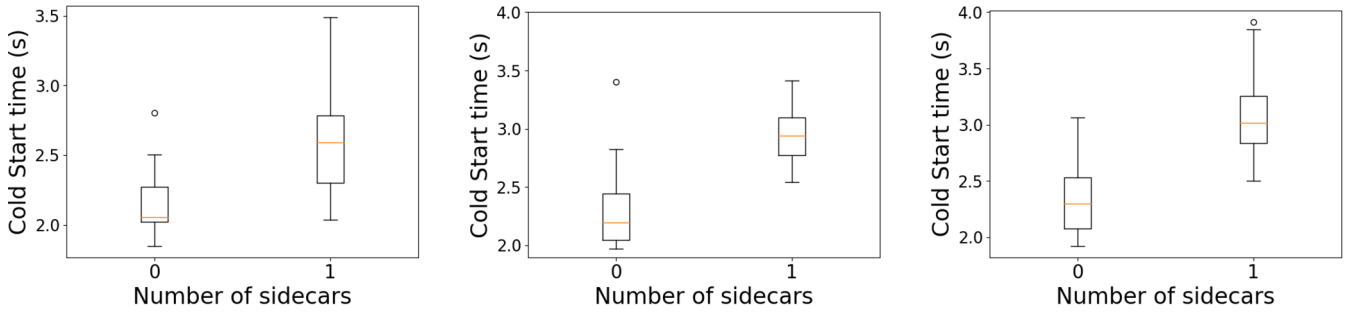
Thus, while we include Envoy in our performance comparisons to illustrate trade-offs, our approach is not intended as a direct competitor for Envoy or as a drop-in replacement for its extensive out-of-the-box features. Instead, it aims to provide modular sidecar containers that can be composed or updated independently. If minimum latency or cold-start time is paramount and existing Envoy filters suffice, Envoy remains a strong candidate.



a) Go implementation

b) Java implementation

FIGURE 9 | Latency depending on the number of proxy sidecars, implemented using Go (left) and Java (right).



a) 8 requests

b) 24 requests

c) 72 requests

FIGURE 10 | Cold-start depending on the number of concurrent requests using Envoy.

Conversely, in scenarios that demand highly specialized logic or nonstandard dependencies, our sidecar approach may prove more flexible even if it entails higher overhead and startup costs.

The experiments in this section have been performed in the same test bed described in Section 5.2.1. In the first experiment we will measure the impact on cold start time when Envoy is used. The Envoy image used has a size of 135 MB. We will measure the cold start time when performing 8, 24, and 72 simultaneous requests. In the more demanding scenario, each worker node must start 18 containers (9 requests per worker node and two containers, Envoy and the application container, per request). Results are shown in Figure 10.

The mean cold start time increases by 20.7% with 8 requests (from 2.144 to 2.588 s), by 28.58% with 24 requests (from 2.281 to 2.933 s), and by 33.99% with 72 requests (from 2.327 to 3.118 s). These times are between the times obtained for 2 and 5 sidecars in Figures 7 and 8. This outcome is reasonable since the image sizes of the proxy sidecars developed in Go were smaller compared to Envoy.

Finally, we have performed an experiment to evaluate the latency introduced by a chain of filters in Envoy. Each filter in the chain just sends the request to the next filter. The experiment was performed with 1, 2, 5, and 10 filters in the chain. Figure 11 shows the results obtained.

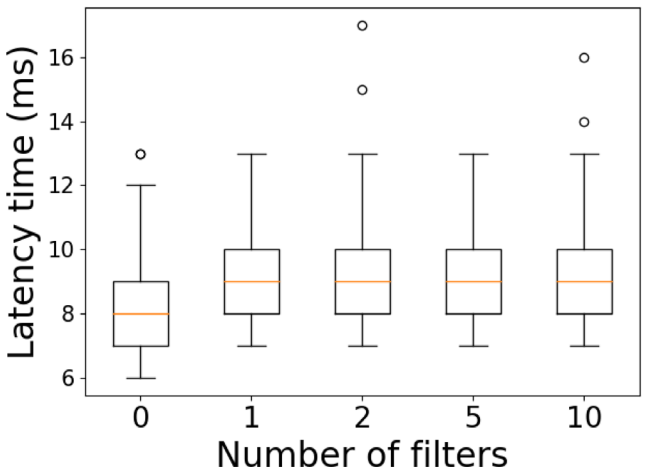


FIGURE 11 | Latency depending on the number of filters in the chain of Envoy.

Table 1 presents a side-by-side comparison of the average cold-start times and steady-state latency metrics for the Go library

TABLE 1 | Comparison of average cold-start times (in seconds) and latency (in milliseconds) for different approaches and numbers of sidecars/filters.

	Cold start (s)				Latency (ms)			
Approach	**1**	**2**	**5**	**10**	**1**	**2**	**5**	**10**
Go library	2.32	2.27	2.74	3.57	9.35	9.91	12.25	16.89
Java library	2.20	3.08	5.47	13.11	8.97	15.55	36.89	72.54
Envoy	3.18	3.18	3.18	3.18	9.65	9.66	9.73	10.09

implementation, the Java library implementation, and Envoy, each tested with varying numbers of sidecars/filters (1, 2, 5, and 10). These measurements showcase the trade-off between flexibility and the overhead introduced by additional containers. Cold-start times correspond to the scenario with 72 concurrent requests.

To cold start time, with up to two sidecars, K8Sidecar with Go and Java libraries actually outperforms Envoy. With five sidecars, the Go implementation outperforms Envoy; however, the Java implementation is worse. With ten sidecars, the Go implementation has an overhead of 12% with respect to Envoy, while the Java implementation shows a big degradation in the cold start time, this degradation is due to the bigger size of the Java image, 60 MB, compared to the Go image, 10 MB.

In terms of latency, with one sidecar K8Sidecar shows a similar behavior as Envoy using the Java and Go libraries. With 5 sidecars, the Go implementation shows an overhead of 25.9% compared to Envoy (just 2.52 ms), while the Java implementation imposes an overhead of 279%. With 10 sidecars, the Go implementation shows an overhead of 67.4% compared to Envoy (just 6.8ms), and the Java implementation induces an overhead of 62.45 ms.

From these results, it can be observed that for up to 1 and 2 filters and nearly 5, the chain-of-sidecars approach (especially using the Go library) remains reasonably close to Envoy in terms of cold start and latency. Overall, these measurements indicate that our approach is not designed for minimal latency or very large sidecar chains, nor does it aim to compete directly with Envoy's suite of built-in features. Instead, the chain-of-sidecars design deliberately prioritizes developer flexibility, allowing sidecars in any programming language with arbitrary libraries, at the cost of increased overhead and cold-start time, especially when the chain grows beyond a few sidecars. If a scenario demands only standard traffic management or load-balancing features already provided by Envoy's built-in filters, then Envoy is likely the superior choice in terms of raw performance. However, for specialized logic beyond the scope of Envoy filters-such as advanced data mesh workflows, our approach enables easy sidecar creation and chaining, even incorporating Envoy itself as one more sidecar if desired.

5.3 | Related Work

The evolution of microservices architectures has required the development of tools and patterns that facilitate service communication, observability, and security. One of the significant patterns that have emerged is the utilization of sidecar proxies, which act as intermediaries for managing requests in microservices.

Kubernetes, from version 1.29, has enabled the use of sidecar containers on a pod by declaring containers in the `initContainers` section with an `OnFailure` restart policy. However, they are not intended to act as proxies and there is no clear mechanism about how to compose and chain them.

Envoy Proxy [6] is a high-performance distributed proxy designed to mediate all inbound and outbound traffic for services in a microservices architecture. It offers dynamic service discovery, load balancing, web protocol support, and observability. Envoy is designed to be transparent to the application, requiring no changes to the code of the services themselves. It can be deployed as a proxy sidecar, handling communication between services, or as an edge proxy, managing inbound and outbound traffic.

Although Envoy Proxy offers a range of functionalities for traffic management and service mesh architectures, our software solution prioritizes a more adaptable and dynamic approach to sidecar management at deployment time specifically tailored for Kubernetes environments. The key distinctions between the two approaches are outlined in Table 2.

In Reference [11], the sidecar pattern is one of the patterns used to propose an architecture for cloud-native applications deployed on Kubernetes. In their proposal the proxy sidecar performs token-based authentication, therefore it is a reusable proxy across different microservices that can be tailored by using `ConfigMaps`.

The work [12] presents a Kubernetes operator (CRDs and their controllers) to launch and monitor workloads in external HPC clusters managed by Slurm, LSF or Ray. When a Custom Resource (CR) is created, the controller starts a pod (a bridge) in Kubernetes that runs the job in the external cluster. Besides the status of the CR is updated when the status of the job in the external cluster changes. The paper provides an example of a bridge for Slurm and an example of a deployment managed by Kubeflow Pipelines, which offers a Python SDK for defining and controlling computational directed acyclic graphs, where each component execution corresponds to a single container execution.

The performance impact of service mesh sidecars has been studied in Reference [13], where the main sources of overhead (IPC, read, write, notification, sidecar protocol parsing, sidecar filter, and processing) were identified. The total performance overheads

TABLE 2 | Comparison of features between Envoy and k8sidecar.

Nr.	Envoy feature	K8Sidecar feature
F1	Employs a singular proxy instance that aggregates all filters, necessitating all traffic to flow through this centralized point.	Opts for a decentralized approach, where each filter operates as an independent proxy. Filters can be managed and scaled independently.
F2	Requires recompilation for most changes, integrating updates directly into the proxy binary. This process ensures consistency but can be less flexible.	Enables specific proxy updates, reducing downtime and accelerating feature deployment.
F3	Extension development is limited to C++ and WebAssembly, restricting the choice of programming languages.	Provides a more flexible environment for proxy development, supporting any language that can interact with HTTP/CloudEvents [10].
F4	Uses own library for extensions, with specific specifications and functions. This requires developers to adapt to Envoy's framework.	Leverages standard HTTP/CloudEvents protocols using conventional library methods.
F5	Has a wide range of use cases supported by an extensive but limited library of extensions. The scope for community contributions is bound by the complexity of Envoy's architecture.	Envisions a "Proxy Store" for future development, aiming to host a repository of open-source proxies created by the community.
F6	Configurations are defined at compilation time, embedding settings directly into the proxy.	Allows for deployment time configuration, enabling on-the-fly adjustments without recompilation.
F7	Integrates all filters within the same container, streamlining the filter chain.	Distributes each filter across different containers.
F8	Employs a strict priority system defined in the configuration, dictating the order of filter processing.	Implements a numerical priority system, offering granular control over filter execution order.
F9	Extensive support for various web protocols, targeting a wide array of networking needs and applications.	Focuses primarily on HTTP and CloudEvents, but other protocols such as AMQP can be implemented.

of a service mesh will be a function of these basic operations. The paper shows that HTTP and gRPC protocol parsing is the main source of overhead. The adequacy of service mesh to mobile edge cloud has been discussed in Reference [14] where DevOps is identified as the main driver of service mesh implementations. In DevOps, the requirements in terms of performance, observability, communication patterns and protocols, and traffic control among others are substantially different from those required in mobile edge cloud. The paper argues that the use of the proxy sidecar incurs both latency and resource overheads. Based on these two works, we can conclude that the chain of sidecars proposed in this work might not be suitable for mobile edge cloud but can be suitable for other IT scenarios allowing rapid experimentation of new features.

6 | Conclusions and Future Work

This paper has introduced K8Sidecar, a Kubernetes software solution that leverages the sidecar proxies pattern to dynamically integrate sidecar containers into microservices and serverless architectures. By intercepting resource creation requests, K8Sidecar not only facilitates modularity at deployment time but also bolsters observability while maintaining the integrity of service logic within the Kubernetes ecosystem. The utilization of custom resource definitions allows the dynamic specification of sidecar configurations, enhancing flexibility and adaptability.

Our study provides a thorough explanation of the proposed architecture, libraries offered to develop new sidecars, and sample use cases such as monitoring, data management, or authentication. Furthermore, measuring the impact on cold start and latency, we have determined that while the number of sidecars should be carefully considered in deployments where cold start performance is critical, their impact on latency remains modest (less than 10 ms, even with 10 sidecars included in a Kubernetes architecture).

Moving forward, the development of K8Sidecar could greatly benefit from exploring the integration of additional communication protocols beyond HTTP and CloudEvents, such as AMQP or MQTT, to broaden its applicability in diverse IoT and real-time data processing scenarios. Further, implementing AI-driven automation within the sidecar management process could enhance the system's ability to predict and adapt to dynamic scaling needs and optimize resource allocation.

Additionally, future studies could investigate the trade-offs between functionality and performance overhead in more depth, particularly in edge computing environments where resources are limited and latency is critical. These advancements could help in refining the K8Sidecar's capabilities, making it even more versatile and efficient for modern cloud-native applications.

Author Contributions

Andoni Salcedo-Navarro: conceptualization, Investigation, Software, Experiments, Writing – original draft, Writing – review and editing. **Miguel Garcia-Pineda:** supervision, Writing – review and editing,

Funding acquisition. **Juan Gutiérrez-Aguado:** conceptualization, Software, Supervision, Writing – review and editing, Funding acquisition.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The authors have nothing to report.

Endnotes

[1] The code repository with the operator is available at https://github.com/cloudmedialab-uv/k8sidecar.

[2] https://github.com/cloudmedialab-uv/k8sidecar-go-lib.

[3] https://github.com/cloudmedialab-uv/k8sidecar-java-lib.

[4] An example of a rate-limiting sidecar, implemented using the Go library, can be found at https://github.com/cloudmedialab-uv/k8sidecar-go-lib/tree/main/examples/ratelimiter.

[5] An example of a logging sidecar using the Java library can be found at https://github.com/cloudmedialab-uv/k8sidecar-java-lib/tree/main/examples/logging.

[6] The source code for this sidecar using the Java library can be found at https://github.com/cloudmedialab-uv/k8sidecar-java-lib/tree/main/examples/authentication.

References

1. N. Dragoni, S. Giallorenzo, A. L. Lafuente, et al., *Microservices: Yesterday, Today, and Tomorrow:195-216* (Springer International Publishing, 2017), https://doi.org/10.1007/978-3-319-67425-4_12.

2. S. R. Goniwada, *Cloud Native Architecture and Design Patterns:127-187; Berkeley* (Apress, 2022).

3. B. Burns and D. Oppenheimer, "Design Patterns for Container-Based Distributed Systems," in *USENIX Association* (Denver, CO, 2016), 108–113.

4. J. T. Duarte Maia and F. Figueiredo Correia, "Service Mesh Patterns," in *EuroPLop '22* (Association for Computing Machinery, 2023), https://doi.org/10.1145/3551902.3551962.

5. L. Giamattei, A. Guerriero, R. Pietrantuono, et al., "Monitoring Tools for Devops and Microservices: A Systematic Grey Literature Review," *Journal of Systems and Software* 208 (2024): 111906, https://doi.org/10.1016/j.jss.2023.111906.

6. Lift, "Envoy Proxy," 2024.

7. K. Hightower, B. Burns, and J. Beda, *Kubernetes: Up and Running Dive Into the Future of Infrastructure*, 1st ed. (O'Reilly Media, Inc., 2017).

8. V. E. Eyk, L. Toader, S. Talluri, L. Versluis, A. Uţă, and A. Iosup, "Serverless Is More: From PaaS to Present Cloud Computing," *IEEE Internet Computing* 22, no. 5 (2018): 8–17, https://doi.org/10.1109/MIC.2018.053681358.

9. B. Ibyram and R. Huss, *Kubernetes Patterns*, 2nd edi ed. (O'Reilly Media, 2023).

10. Cloud Native Computing Foundation, *Cloudevents Project* (2024), https://cloudevents.io/.

11. Q. Jiao, B. Xu, and Y. Fan, "Design of Cloud Native Application Architecture Based on Kubernetes," in *IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)* (IEEE, 2021), 494–499, https://doi.org/10.1109/DASC-PICom-CBDCom-CyberSciTech52372.2021.00088.

12. B. Lublinsky, E. Jennings, and V. Spišaková, "A Kubernetes Bridge Operator Between Cloud and External Resources," in *IEEE 8th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA)* (IEEE, 2024), 263–269, https://doi.org/10.1109/icccbda56900.2023.10154770.

13. X. Zhu, G. She, B. Xue, et al., "Dissecting Overheads of Service Mesh Sidecars," in *15th ACM Symposium on Cloud Computing (SoCC)* (Association for Computing Machinery, 2023), 142–157, https://doi.org/10.1145/3620678.362465.

14. A. Duque, C. Klein, J. Feng, X. Cai, B. Skubic, and E. Elmroth, "A Qualitative Evaluation of Service Mesh-Based Traffic Management for Mobile Edge Cloud," in 2022 22nd *IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)* (IEEE, 2022), 210–219, https://doi.org/10.1109/CCGrid54584.2022.00030.